MASTER'S THESIS · RESEARCH AND IMPLEMENTATION STATUS

Hybrid photonic sorting

Complete master's thesis documentation

Literature review, mathematical models, implementation, and evaluation

Bharadwaj Srikumar

M.Sc. Computer Science · Bochum University of Applied Sciences

Supervisor: Prof. Dr. Henrik Blunck

Research status: 24 October 2026

Working title

Mapping Hybrid Optoelectronic Sorting Architectures onto Modern Photonic Accelerators: A Hardware-Aware Evaluation and Simulation

This document describes the current state of the thesis. It connects literature, mathematical derivations, executable code, and saved results. Physical hardware validation, photonic runtime advantages, and energy efficiency remain unresolved.

Contents

Abstract 3

1. Research question and scope 4

2. Foundations: from numbers to optical signals 5

3. Literature review and consequences for mapping 8

4. Complexity model C = (T, S, H, D) 11

5. The five sorting architectures 14

6. Mathematical mappings and stable records 16

7. Q.ANT SDK and periodic comparison 20

8. Reference experiment: inputs, quantization, and noise 23

9. A shared signal and noise model 25

10. Quality metrics: complete order and useful ranking 28

11. Results from saved experiments 30

12. Calls, data movement, and potential system costs 34

13. From concept to executable code 36

14. Reproducibility and verification 37

15. Hardware questions and outstanding work 38

16. Overall assessment of the current state 40

Appendix A. References and their roles 41

Appendix B. Source corpus and result provenance 47

Appendix C. Terminology and reading guide 49

Abstract

This thesis investigates the accuracy and resource conditions under which complete, stable sorting can be usefully mapped onto hybrid photonic processors. The complete execution path matters: photonic processing, discrete decisions, ties, data movement, and electronic output.

Of five architectures reviewed, Bitonic and rank sorting are studied in depth. Bitonic processes relatively few comparisons in dependent stages; rank sorting groups a quadratic pair-comparison phase. The unchanged model C = (T, S, H, D) separates total scalar sequential work, logical storage, installed processing positions, and unregenerated optical depth. T is a real-RAM-like operation count, not physical time in seconds.

Saved inputs, exhaustive 4-/8-bit pair checks, and complete sorting runs support mathematical and CPU-level functionality. The periodic SDK component with host-side phase calculation makes correct decisions after reference correction. However, its input difference already resides on the host in the existing path; the additional call has no demonstrated benefit for that decision.

A separate affine phase calculation inside the MVM is mathematically equivalent but loses precision in the tested CPU backend: 40 of 65,536 ordered 8-bit pairs become false ties without added noise. The public interface also returns this intermediate value to the host. This finding limits the specific variant; it does not establish a general hardware limit.

Additional quality metrics distinguish complete ordering from a useful approximate ranking: with 8 bits, N = 256, distinct keys, and $\eta = 0.25$, only 5.9% of Bitonic outputs in the reference model are completely correct, yet mean Kendall-τ-b is 0.999826 and Recall@10 is 99.88%. Errors predominantly involve adjacent keys.

The shared noise model makes signal spacing, quantization, and disturbance location comparable. Its parameters are sensitivity assumptions, not a Q.ANT calibration. A digital CPU baseline is available; GPU, NPU, and energy measurements remain pending. Outputs for all 288 current configurations are available in full and independently rescored. Rank sorting consistently uses the complete Louri matrix with N^2 separately evaluated entries.

Table 1. Current status and limits of the evidence

Area	Available evidence	Outstanding
Literature and model	Five architectures; C = (T,S,H,D); derived bounds	No new universal complexity theory
Implementation	Bitonic, rank sorting, linear and periodic CPU SDK	Useful cascade without a host round trip
Quality and noise	Reference, τ, distance errors, recall; shared controls	Device-specific transfer characteristic, noise, and drift
Performance	Stable digital CPU baseline	GPU, complete NPU timing, and energy

1. Research question and scope

1.1 Why sort on photonic processors?

Sorting is a fundamental data-processing operation. It combines a simple ordering requirement with many comparisons and data movements. This makes it suitable for testing whether a photonic compute core can contribute usefully beyond dense neural matrix operations. A kernel that can be expressed mathematically is not yet an efficient system mapping: its contribution and the cost of integrating it into the program are decisive.

Bitonic has a fixed, data-independent comparison schedule. Rank sorting can prepare its pair comparisons independently. Both therefore initially appear amenable to parallelization. However, their requirements differ: Bitonic needs relatively few comparisons across several dependent stages, whereas rank sorting needs substantially more comparisons but can group them into a wide operation. This comparison forms the core of the thesis. [6-10]

The central question is: **Under which accuracy, data-movement, and resource conditions can Bitonic and rank sorting be usefully mapped onto hybrid photonic processors?** A negative result, such as interfaces or electronic processing eliminating the benefit, is a scientifically relevant answer.

1.2 Specific research questions

- Which parts of a stable comparison and complete sorting operation are linear transformations, nonlinear decisions, accumulation, or data movement?
- What conclusions do the five historical or hybrid architectures support under common definitions of T, S, H, and D?
- How do signal spacing, quantization, noise, and ties affect complete sorting correctness and approximate ranking quality?
- Which calls, logical data volumes, and host operations does an SDK mapping require compared with a direct electronic decision?
- Which hardware properties would be necessary for a complete hybrid execution to match or outperform the measured digital baseline?

1.3 What the thesis investigates

The literature review covers Analog Photonic Computing, programmable photonic circuits, hybrid precision, conversion, noise, regeneration, energy, and five sorting architectures. The executable investigation focuses on integer 4-/8-bit keys, Bitonic, and rank sorting. The experiments examine mathematical controls, CPU software integration, synthetic signal models, and a digital CPU baseline.

Q.ANT provides a concrete reference for a publicly inspectable SDK path with periodic nonlinearity. An Envise-like separation of photonic matrix operations and digital postprocessing provides an architectural contrast. Historical sorters are evaluated functionally; their optical components, spatial layouts, and clock networks are not reconstructed as digital twins.

Energy superiority and unrestricted cascading are not assumed. Chip development and universal complexity proofs are outside the scope. PRISM addresses a related selection task, not complete stable sorting. [23]

2. Foundations: from numbers to optical signals

2.1 Analog, digital, and photonic describe different properties

Digital representations encode values using discrete symbols. Analog processing maps values to continuous physical quantities such as voltage, field amplitude, phase, or power. Photonic describes the physical medium. A photonic system may transmit digital optical bit sequences, compute analog weighted sums, or combine both with electronics.

These terms must therefore not be treated as synonyms. Some historical sorters studied here use bit-serial optical data and electronic smart pixels. The modern comparison model encodes whole keys as normalized numbers. An FPGA radix sorter with optical links sorts electronically even though its communication is photonic. [7-11]

A real-RAM model is a mathematical idealization with exact scalar operations. It does not assert infinite resolution in real hardware. Complexity analysis and finite numerical experiments answer different questions and are treated separately.

2.2 What light offers for computation

Optical systems can carry many spatial or spectral channels simultaneously. Interference, weighted transmission, and detection enable linear transformations. Wavelength-division multiplexing uses multiple carriers; spatial broadcasting distributes a signal to multiple locations. These properties can support wide operations but require appropriate sources, modulators, receivers, and data supply. [1, 2, 28]

A generic linear core is described mathematically as $y = Wx$. In a coherent system, complex field amplitudes can interfere; subsequent power detection is not automatically a measurement of a signed real value. In incoherent architectures, weighted intensities or photocurrents are summed. Positive and negative weights require explicit encoding or differential readout. The architecture determines where summation takes place.

Table 2. Generic hybrid signal path

Step	Operation	System quantity to assess
1. Electronic input	Read, scale, and pack keys	Memory and host time
2. Electro-optic encoding	Map values to phase / amplitude	Resolution, modulation, and energy
3. Photonic core	Linear transformation / confirmed nonlinearity	Transfer characteristic, loss, and disturbance
4. Readout	Detection, conversion, and transfer	Noise, grid, and interface
5. Electronic output	Decide, exchange / accumulate, validate	Complete sorting quality and time

Our schematic, not a Q.ANT layout. Internal boundaries may differ in a confirmed cascade.

The diagram illustrates a generic hybrid signal path, not a reconstruction of the Q.ANT chip. An optical core does not operate in isolation: inputs are prepared, converted, processed, read out, and reused electronically. Memory and control remain within the system boundary. [1, 12, 22]

2.3 Interference as an example of a periodic transfer function

A change in refractive index changes the optical phase. For a path of length L and wavelength λ, a simple model gives:

$$\Delta\varphi = \frac{2\pi L\,\Delta n}{\lambda}$$

At one output of an ideal symmetric, lossless interferometer:

$$P_{\text{out}} = \frac{P_{\text{in}}}{2}\left[1 + \cos(\Delta\varphi)\right]$$

Within its linear operating region, a photodiode produces I = ℛP, where ℛ is its responsivity. Measured power is nonnegative. A difference between two positive photocurrents can nevertheless produce a signed signal. The periodic power characteristic explains why a nonlinear function may be physically accessible. It is not automatically ReLU or a robust threshold decision.

Lithium niobate enables fast electro-optic phase modulation. The TFLN system investigated in [16] is a separate research setup; it does not establish specific Q.ANT input limits or freely cascaded Q.ANT operations. The native SDK function tcos must also not be equated with the ideal cosine used in the explanation above.

2.4 APC as a programmable computing concept

Macho-Ortiz et al. describe Analog Programmable-Photonic Computation based on programmable integrated photonics. Its elementary information unit, the anbit, uses two complex amplitudes. Linear transformations are organized into classes of programmable gates; feedback extends combinational systems into sequential ones. [2, particularly the sections on anbits and gates]

The similarity between a two-component representation and quantum formalisms does not make APC a quantum computer. The signals used here are classical optical quantities. Likewise, a general APC gate formalism does not guarantee implementation of every sorting component in a commercial SDK. APC provides the physical and mathematical context for this thesis; the concrete implementation is tied to verifiable API functions.

2.5 Loss, crosstalk, noise, and precision

Loss attenuates the signal. An ideal passive path with constant transmission factor a per stage has power $P_0 a^D$ after D stages. This is initially a power model. Signal-to-noise scaling depends on where noise is introduced, amplified, correlated, or changed by regeneration. No universal linear or geometric SNR decline is assumed.

Coherent crosstalk can interfere constructively or destructively depending on phase; it is not exclusively attenuation. Incoherent interference may form a background. Quantization describes discrete representation and decision thresholds. Random noise, systematic offsets, and drift must be distinguished from it. Reliability literature motivates treating these effects separately. [1, 21, 30, 33, 34]

Key width, storage format, quantization step, and ENOB describe different quantities. A BF16 array establishes a software format, not 16 effective analog bits. A simulated step Δ is a modelling choice; only a defined measurement can establish physical resolution or effective bit count.

2.6 Regeneration and optical depth

Regeneration restores a data signal and thereby limits the length of an unregenerated segment. An O/E/O boundary can provide this through detection, electronic processing, and re-emission. Optical regeneration is also studied, for example through nonlinear signal processing. Such techniques require additional components and operating conditions. [20]

Electronic control alone does not regenerate the optical data signal. Conversely, execution without a host round trip may already include internal conversions and electronic memory. Determining D requires knowledge of the actual data-signal path. D cannot be inferred from a Python function, an SDK call count, or a marketing description.

3. Literature review and consequences for mapping

3.1 Approach and types of evidence

The review connects original papers on sorting networks, physical computing models, and hybrid photonic systems. The supplied corpus contains 25 PDFs, including an extensive research compilation and a collection of optical sorting papers. These compilations are not counted as additional independent experiments. The two presentations provide the thematic context; technical claims are checked against original sources and code. The corpus mapping and supplementary literature are listed in the appendix.

Four types of statements are distinguished: mathematical derivations, published device or demonstrator findings, our CPU/simulation evaluations, and explicit assumptions. A projected scale-up in a source remains a projection. A neural-network accuracy result does not establish error-free sorting of adjacent keys. Our own derivations are identified as such.

3.2 System integration beyond an isolated optical operation

Stroev and Berloff [40] discuss analog photonic platforms particularly for inference and optimization. Their review does not establish a general zero error rate for sorting. Error-free ideal computation must not be adopted as a physical property.

McMahon [1] explains which optical properties may enable computational advantages and why no individual property guarantees them. For this thesis, the implication is to consider data preparation, achievable precision, and integration into the complete program alongside light-propagation time.

Ahmed et al. [12] describe an integrated hybrid processor with four photonic 128×128 cores and digital control. The power breakdown reported in the source is 43.7% logic/NoC/SRAM, 35.6% data converters, 16.4% photonic cores, and 4.2% PCIe. These demonstrator figures illustrate the importance of peripheral systems; they are not cost parameters for our Q.ANT sorter. Nonlinearities and accumulation are digitally integrated in that architecture.

Farmakidis et al. [32] classify neural photonics partly by the position of optical-electronic boundaries. Al-Qadasi et al. [33] examine scaling limits through optical and electrical link budgets. Both perspectives motivate explicit execution boundaries in this mapping: high optical parallelism is useful only if input supply, readout, and postprocessing can keep pace.

3.3 Hybrid precision

Meng et al. [13] combine digital and analog processing to obtain higher numerical precision. In LightMat-HP, Gong et al. [17] investigate low-bit photonic suboperations and digital reconstruction for GEMM. Giamougiannis et al. [34] combine limited core dimensions with tiling and task-dependent precision.

The shared implication is not a free accuracy improvement: decomposition, repetitions, accumulation, and additional conversion must all be counted. The smallest relevant key spacing is particularly critical for sorting. A small mean matrix error can still reverse a comparison at a decision boundary. Ties and adjacent keys are therefore tested explicitly here, rather than measuring only a global mean computation error.

3.4 Calibration and selective digital processing

Zhao et al. [14] combine offline configuration with online adjustment for programmable optical processors. The HOOC approach shows that configuration errors and subsequent correction require separate processing steps. A measured transfer characteristic cannot be assumed to remain permanently exact without calibration.

HyAtten [15] investigates selective processing of problematic values on a digital path, avoiding high-resolution conversion for every value. This motivates a possible future sorting variant in which uncertain comparisons are checked electronically. That variant is not implemented in the current sorter code. Evaluating its benefit would require accounting for detection cost, missed boundary cases, and the fraction of digital rework. The current experiments do not use a corrective sort after execution.

3.5 Native nonlinearity and optical conversion

Sakata [18] investigates photonic analog-to-digital conversion with nonlinear Fabry-Perot resonators. Nauman et al. [19] study a photonic ADC architecture supporting MAC operations. Research on PCM-based converters [31] also addresses the interface between representation and processing. These papers demonstrate alternative technical approaches; their resolution, energy, or thresholds are not transferred to Q.ANT.

Hu et al. [16] demonstrate TFLN-based computation with electro-optic modulation; their applications also include electronic activations. Xu et al. [20] review optical signal processing and regeneration. Zhou et al. [35] investigate quantum-emitter-based nonlinear activations in a different architecture. None of these findings establishes cascading of the difference and periodic functions used in our SDK without a host round trip.

3.6 Noise modelling as a structural contribution

López-March et al. [21] first model physical disturbances as photocurrent fluctuations and propagate them through the receiver into the APC representation. The source separates thermal noise, shot noise, and relative intensity fluctuations, among other effects. Its measured silicon PIP arrangement is not our Q.ANT system. We therefore adopt the separation of disturbance locations, units, and error propagation, rather than a supposedly universal σ.

For our comparisons, signal spacing and its associated disturbance must use the same unit. “Equal noise strength” is incomplete without the interface, normalization, and quantization. The shared model in Chapter 9 implements this requirement directly.

3.7 Energy efficiency: supported conclusions

Bahr, Steinmeyer, and Pernice [22] analyze conversion, weight setting, memory access, and optical losses at system level. Large dense matrix operations can distribute conversion costs over many MACs. However, the cost model explicitly excludes digital control logic; it therefore does not provide a complete measurement of our sorting program. Weight reuse, optical loss, and data supply are essential conditions.

The source's comparison table also includes a B100 reference point; the blanket claim that this version considers only old digital systems is therefore incorrect. The decisive limitation remains: peak or literature values at specified precisions do not replace a CPU/GPU sorting benchmark with the same task. A narrow two-input difference kernel cannot automatically exploit the amortization of a large dense MVM. [22, Section III, Table II and Table V]

This establishes a testable sequence for the thesis: functional mapping, numerical reliability, complete runtime, and then energy within the same system boundary. Without measured power consumption, no energy gain is calculated from a manufacturer's GOPS figure.

3.8 Why PRISM targets a different task

PRISM [23] computes photonic similarity scores for KV-cache block selection and combines them with electronic top-k selection. Its objective is useful approximate preselection within inference, not complete stable sorting of all values. It is therefore treated as a related approach to a more tolerant selection task.

Recall@k supplements our evaluation by showing the selection consequences of small rank errors. This does not make the experiments a PRISM benchmark: its inputs, score distributions, memory accesses, and downstream model quality are absent. High recall of the largest integer keys alone does not justify a claim about LLM inference.

4. Complexity model C = (T, S, H, D)

4.1 Common definitions

T: total sequential work of a complete sort. Scalar arithmetic operations, comparisons, logical decisions, memory accesses, and necessary record movements are counted with constant cost per model step. Exact real arithmetic is an idealization. Input, initialization, accumulation, validity checking, and output are included; a parallel stage is expanded into its scalar operations.

S: maximum simultaneously required logical storage for one batch. Data and auxiliary state are counted in logical words. A key, index, or counter counts as one word in the model. This is not bit complexity. In a finite implementation, these objects have specific formats and byte costs, documented separately.

H: installed processing positions at word granularity. Granularity is kept fixed across architecture rows. H is neither the number of Python calls nor total chip area. Evaluating the area of memory cells, wiring, optics, drivers, and converters requires an additional physical layout model.

D: maximum number of consecutive optical stages without regeneration of the data signal. D ends at an actual restoration of the data signal. Reusing a regenerating loop does not automatically increase the depth of a single unregenerated segment.

The tuple remains unchanged. Seconds, throughput, energy, signal spacing, error rates, and call counts are supplementary evaluation quantities, not new complexity coordinates.

4.2 Why parallel architectures can have the same T

A Bitonic stage performs N/2 independent compare-and-exchange operations. In a spatial pipeline these may execute simultaneously, but they still contribute $\Theta(N)$ work to the real-RAM-like T. Both pipelined and recirculating architectures require dependent stages. “Pipeline = sequential, recirculation = fully parallel” would therefore be an incorrect distinction.

Differences include installed hardware, overlap of multiple batches, serialization, utilization, and physical clocking. Equal asymptotic work implies neither equal latency nor equal throughput. A pipeline's output interval can be shorter than its time to the first complete result.

4.3 Historical models as reference points

Thompson [3] describes physical area and time in a VLSI model and derives $AT^2 \geq N^2/16$ for the DFT under his assumptions. This concerns spatial resources and communication. It is not a sorting lower bound, and A is not identical to our S or H.

Caulfield [4] relates normalized spatial, temporal, and fan-in resources to problem complexity. The relation $X_A X_T X_F \geq X_P$ explains why a wide optical reduction cannot be assessed solely by counting abstract stages. The source's time quantity is not our sequential T.

Li and Monticone [5] connect the structure of linear operators with physical space requirements and communication. This motivates attention to nonlocality and routing. However, a result for a linear operator cannot be applied unchanged to the complete nonlinear sorting task. Our tuple is an operational comparison framework, not a law copied unchanged from these three papers.

4.4 Lower, upper, and tight bounds

Ω denotes an asymptotic lower bound, O an upper bound, and Θ matching lower and upper bounds. These notations do not mean best, worst, and average case. The following tables concern the specified architectures and representations, not optimal sorting over every possible method. For N ≥ 2, all logarithms are base two.

Table 3. Lower bounds of the specified architectures

Architecture	T	S	H	D
Pipelined Bitonic	Ω(N log²N)	Ω(N)	Ω(N log²N)	Ω(log²N) / Ω(1)
Beyette, recirculating	Ω(N log²N)	Ω(N)	Ω(N)	Ω(1)
Desmulliez, shuffle	Ω(N log²N)	Ω(N)	Ω(N)	Ω(1)
Louri, materialized ranks	Ω(N²)	Ω(N²)	Ω(N²)	Ω(1)
Radix, fixed parameters	Ω(N)	Ω(N)	Ω(1)	Ω(1)

Our derivation under the definitions and assumptions in Chapter 4. Pipeline D: passive / regenerated after every stage.

Table 4. Tight bounds of the specified architectures

Architecture	T	S	H	D
Pipelined Bitonic	Θ(N log²N)	Θ(N)	Θ(N log²N)	Θ(log²N) / Θ(1)
Beyette, recirculating	Θ(N log²N)	Θ(N)	Θ(N)	Θ(1)
Desmulliez, shuffle	Θ(N log²N)	Θ(N)	Θ(N)	Θ(1)
Louri, materialized ranks	Θ(N²)	Θ(N²)	Θ(N²)	Θ(1)
Radix, fixed parameters	Θ(N)	Θ(N)	Θ(1)	Θ(1)

Our derivation under the definitions and assumptions in Chapter 4. Pipeline D: passive / regenerated after every stage.

Table 5. Upper bounds of the specified architectures

Architecture	T	S	H	D
Pipelined Bitonic	O(N log²N)	O(N)	O(N log²N)	O(log²N) / O(1)
Beyette, recirculating	O(N log²N)	O(N)	O(N)	O(1)
Desmulliez, shuffle	O(N log²N)	O(N)	O(N)	O(1)
Louri, materialized ranks	O(N²)	O(N²)	O(N²)	O(1)
Radix, fixed parameters	O(N)	O(N)	O(1)	O(1)

Our derivation under the definitions and assumptions in Chapter 4. Pipeline D: passive / regenerated after every stage.

For the pipeline, the first D alternative denotes passive propagation and the second regeneration after every stage. For rank sorting, S applies to the materialized matrix representation. For radix sorting, key width, radix, pass count, and number of processing streams remain constant; H excludes growing data storage. These assumptions form part of each bound.

4.5 Deriving the five rows

For N = 2^m, the compact Bitonic network has stage count K and comparison count C_B:

$$K = \sum_{j=1}^{m} j = \frac{m(m+1)}{2}, \qquad C_B = \frac{NK}{2}$$

Each comparison requires a positive constant number of scalar steps. The complete fixed schedule therefore establishes both Ω(NK) and O(NK). Input and output add Θ(N). The fixed shuffle schedule requires P = $m^2 - m + 1$ passes, including bypass; moving N words in each pass also leaves the order at Θ(N $\log^2 N$). [6-9]

One live batch and a constant number of reused buffers justify S = Θ(N) for the modelled Bitonic executions. A spatial design installs Θ(NK) comparison positions; reused arrays install Θ(N). Passively cascaded data traverse Θ(K) stages. Regenerating boundaries limit D to a constant segment length.

The materialized rank architecture processes a quadratic comparison layer. Broadcasts, differences, thresholds, rank summation, and selection require a constant number of quadratic passes in the scalar model. T, S, and H are therefore each Θ(N^2). A fixed number of wide optical layers has D = Θ(1), without implying constant optical power or area. A streamed electronic rank implementation could use less memory; it is not the matrix representation defined here. [10]

Stable radix sorting with q passes and R buckets costs Θ(q(N+R)). With fixed word format and radix, this reduces to Θ(N). Digit extraction is explicitly allowed as a constant-cost integer operation. This row does not claim linear comparison-based sorting of arbitrary real values. Fixed processing streams yield H = Θ(1), while growing data storage requires S = Θ(N). [11]

The tables describe idealized architecture families. In the actual Python code, precomputed pair schedules, batch vectors, and intermediate copies can occupy additional memory. In particular, the Bitonic code stores the comparison schedule; its actual memory use is not guaranteed by the idealized single-batch bound alone. These implementation costs must not be inferred from the table.

5. The five sorting architectures

5.1 Spatial pipelined Bitonic by Stirk and Athale

The architecture arranges compare-and-exchange modules in successive stages, connected by fixed optical channel permutations. A bitonic merge combines an ascending and a descending subsequence. Merge depths from 1 to m form the complete sorter. Pairs within a stage are independent; stages depend on one another's data. [7, Section II]

The paper examines technologies for comparison, latching, and exchange. For D, the distinction between regenerative forwarding and passive propagation of the data signal is decisive. An optical interconnection structure alone does not establish an error-free cascade. Analog fluctuations, dynamic range, and switching thresholds can limit usable depth. [7, particularly Figs. 5-6]

The modern implementation adopts the logical sequence of active C&E stages. Integer records are not represented as the original bit-serial optical sequences. The software schedule gathers pairs, computes differences, and exchanges original records electronically. The source supports the structure, not the speed or energy of this adaptation.

5.2 Recirculating Bitonic by Beyette et al.

Beyette et al. repeatedly use two smart-pixel arrays. Data travel optically between them; electronic comparison logic, latching, and synchronization support processing. Reuse reduces installed hardware relative to a fully unrolled pipeline. The pipeline, in turn, can overlap multiple batches more extensively. [8, Sections 2 and 3]

The source distinguishes system capacity, response time, and output interval. In its comparison with bit-serial input/output, both complete batch executions require $P + b - 1$ clock cycles under the stated assumptions; output intervals are b for the pipeline and P for recirculation. This specific finding must not be generalized into equality of all physical runtimes. [8, Section 3.C]

For $N = 16$, $m = 4$ and $P = 13$. With $b = 8$, that source model produces a complete response after 20 cycles. Electronic restoration between transfers limits D for each optical segment. More loop passes increase work and traffic, not automatically the unregenerated depth.

5.3 Perfect-shuffle Bitonic by Desmulliez et al.

This architecture uses a reused sorting-node array, memory, and configurable control masks. A perfect shuffle changes channel assignments. Nodes compare, exchange, or bypass data. Bit-serial processing starts with the most significant bit; additional cycles load masks and reset the system. [9, Sections 2-5]

The fixed shuffle schedule has $P = m^2 - m + 1$ steps. The paper's cycle accounting uses $(b + 2)P$: for $N = 16$ and $b = 8$, this is 130 cycles. It counts different hardware operations from the compact active Bitonic implementation. Equal asymptotic T therefore does not mean that the two sources predict equal cycle counts.

This architecture illustrates the trade-off between nonlocal optical interconnection and electronic state management. The current functional mapping does not present separate Desmulliez results as an independent photonic measurement series. Running the same Bitonic ordering schedule under different architecture names in software would not constitute an independent hardware comparison.

5.4 Broadcast-and-compare by Louri et al.

Louri et al. distribute inputs into a wide comparison structure. Pairwise ordering results are combined into ranks, which determine output positions. Additional handling of equal keys makes rank assignment unique. Smart pixels also contain electronic processing. Describing the design as an entirely optical difference decision would therefore overstate the evidence. [10, Eqs. 3-7 and Section 3]

The constant number of wide source operations requires comparison area that grows with N and suitable wide reductions. In the scalar real-RAM-like model, this work expands to $\Theta(N^2)$. For the source example [7, 8a, 2, 8b, 5], the one-based ranks are [3, 4, 1, 5, 2] and the stable output is [2, 5, 7, 8a, 8b]. Our zero-based convention describes the same ideal order.

Historical timing calculations in the project materials reproduce a total of 60.2 ns for a complete cycle and 36.8 ns as the period. These are not NPU measurements. Moreover, the cited 10 pJ and 1.5 mW imply approximately 6.67 ns through E/P, whereas the text uses approximately 13.33 ns. This discrepancy remains unresolved; it is not replaced with an apparently precise modern runtime.

5.5 In-network radix sorting by Mizutani et al.

Mizutani et al. combine electronic sorting cores and HBM on multiple FPGAs with optical memory-to-memory links. A 32-bit key is processed in eight 4-bit counting-sort passes. Stable distribution, memory traffic, and network structure are central. Photonics provides communication; it does not compute the comparison decision of an analog C&E module. [11]

The source reports 37.2 GB/s for 16 GiB of data and 9.2 MB/J on the eight-FPGA setup under the stated conditions. Scaling to 256 FPGAs at 983 GB/s for 512 GiB is a model-based scale-up analysis, not an equivalent measured 256-FPGA demonstrator. These evidence types remain distinct.

This architecture matters because it shows that photonics can contribute through data movement as well as analog arithmetic. Its large data volumes, integer operations, and hardware platform are not equivalent to our small CPU lists. It provides an architectural contrast, not a directly applicable performance baseline.

5.6 Selection for the modern evaluation

Table 6. Role of the five architectures in this thesis

Architecture	Adopted idea	Use in the current experiment
Stirk / Athale	Fixed active C&E stages	Functional Bitonic mapping
Beyette	Reused comparison array	Resource and latency contrast
Desmulliez	Shuffle, masks, and bypass	Source-based accounting, no separate NPU run
Louri	Wide pair phase and ranks	Functional rank mapping
Mizutani	Photonic data movement	Hybrid architectural contrast

The two core mappings isolate the contrast between narrow dependent stages and a wide pair-comparison phase. The other architectures inform hardware reuse, routing, and communication costs. This connects the literature comparison to executable evaluation without implying five device implementations that do not exist.

6. Mathematical mappings and stable records

6.1 Defining the task precisely

An input list contains N original records with integer key x_i and original index i. A complete output must contain every record exactly once and arrange the keys in ascending order. For equal keys, original indices must remain ascending. The ideal reference is therefore the lexicographic order (x_i, i).

The analog model supplies only a comparison signal. Keys and indices remain stored as unchanged original data. This separation is crucial: a correct sign suffices for the record-order decision, whereas a distorted difference cannot reconstruct exact minimum and maximum values.

6.2 A linear difference and a nonlinear decision

For B independent pairs, X is a B×2 matrix. The weight matrix has shape 1×2:

$$X_{p,:} = (a_p, b_p), \qquad W = (1, -1), \qquad Y = XW^{\mathsf{T}}$$

Each row produces $d_p = a_p - b_p$. This is linear. Deciding which original record belongs on the left, right, or a particular rank is nonlinear. A pure MVM therefore does not implement complete sorting.

For exact, unclipped values, r = max(d, 0), min(a,b) = a − r, and max(a,b) = b + r. This ReLU identity explains a possible decomposition, not where it executes. ReLU runs on the host in the inspected SDK. [36]

A counterexample prevents misuse of the saturated comparator: at 8 bits, d for (255,0) is 255/256. The sign-oriented reference may clip this to 127/256. Its sign remains correct, but ReLU reconstruction would produce 128 and 127 instead of 0 and 255. The sorter therefore exchanges only original records.

6.3 Bitonic: comparison schedule and exchange

The compact Batcher schedule doubles the group size from 2 to N. Within each group, the partner distance is halved. The partner j = i XOR distance determines the connection; i < j prevents duplicate pair processing. The group determines ascending or descending direction. The current runner uses powers of two and rejects other lengths for Bitonic.

Table 7. The same comparison function in two complete execution paths

Phase	Bitonic	Rank sorting
Prepare pairs	N/2 pairs in the current stage	All N^2 matrix entries, including the diagonal
Difference and decision	Parallel within a stage	Separate decision for each directed cell
Electronic postprocessing	Exchange original records	Subtract lower-triangular bias, threshold, sum columns
Next step	Next dependent stage, up to K	One output from valid ranks

The differences concern dependencies, resources, and data movement; both mappings use the same stable-output contract.

At each stage, the Bitonic data path proceeds from electronic pair gathering to the comparison signal and back to electronic exchange. Only that result can enter the next stage. A batch can group many independent lists, but this does not remove dependencies within a list.

For measured signal đ, a is ordered after b when đ > 0, or when đ = 0 and a has the larger original index. The comparison uses the original indices travelling with the records, not current wire positions. Descending network stages reverse this same total order. Without disturbances, the complete output is therefore stable.

A complete example for [10,3,9,1] with 8-bit normalization:

Table 8. Bitonic example for [10,3,9,1]

Stage	Pairs and direction	Differences	State afterwards
1	(0,1) ascending; (2,3) descending	7/256; 8/256	[3,10,9,1]
2	(0,2) ascending; (1,3) ascending	−6/256; 9/256	[3,1,9,10]
3	(0,1) ascending; (2,3) ascending	2/256; −1/256	[1,3,9,10]

Our illustration based on the derivation or sources cited in this section.

The final original-index sequence is [3,1,2,0]. Even with erroneous decisions, electronic exchanges preserve a permutation. It may nevertheless be incorrectly ordered or unstable.

6.4 Louri rank sorting: the full matrix

The thesis uses Louri as its canonical rank architecture. All N² entries, including both comparison directions and the diagonal, are evaluated separately. For N = 4, this means 16 comparison positions: twelve directed comparisons of different records and four self-comparisons. No reverse result is inferred from a decision already made.

Row i denotes the competing record; column j denotes the record whose rank is determined. For integer keys with step $\Delta = 2^{-b}$:

$$D_{ij} = \Delta(x_j - x_i), \qquad U_{ij} = \mathbf{1}[i > j], \qquad D'_{ij} = D_{ij} - \Delta U_{ij}$$

$$R'_{ij} = \mathbf{1}[D'_{ij} \geq 0], \qquad r_j^{(1)} = \sum_i R'_{ij}, \qquad r_j^{(0)} = r_j^{(1)} - 1$$

The auxiliary matrix U contains ones below the diagonal and zeros elsewhere. Its bias is scaled to the same grid as the keys. For equal keys, it removes the later competitor's contribution to the earlier record's rank. Positive integer differences remain nonnegative after subtracting one grid step. The tie rule is therefore stable.

Without errors, each diagonal entry contributes one, so source ranks start at 1. For [3,1,3,0], the threshold matrix is:

Table 9. Worked example

Competitor i / key	Column 0: 3	Column 1: 1	Column 2: 3	Column 3: 0
0 / 3	1	0	1	0
1 / 1	1	1	1	0
2 / 3	0	0	1	0
3 / 0	1	1	1	1
Column sum / source rank	3	2	4	1

Our illustration based on the derivation or sources cited in this section.

The zero-based positions [2,1,3,0] produce [0,1,3_first,3_second]. The binary threshold decisions are summed, not the raw differences. All original keys and record identities are preserved electronically. Duplicate positions or positions outside 0 to N−1 invalidate the output; they are neither clipped nor repaired by digital re-sorting.

6.5 Fidelity to the source and independent verification

The published example [7,8,2,8,5] yields one-based ranks [3,4,1,5,2]. Two passages on page 3089 contradict the illustrated tie direction or triangular half. The implementation follows the equations and example matrices. [10, Eqs. 1-7]

The direct equation reference and operational sorter are independently checked against stable digital sorting: all 256 ordered 4-bit pairs, 65,536 ordered 8-bit pairs, 65,536 four-element lists over 4-bit keys, and all 24,000 saved lists. All 155,328 inputs agree without noise, both with and without the additional model quantizer. These overlapping control sets are not independent samples. [D8]

6.6 Implementation through linear pair operations

rank_sort prepares the full matrix as N^2 pair rows in row-major order. Cell (i,j) receives x_j and x_i. Weights [1,−1] produce the normalized difference. The host then subtracts ΔU, applies the threshold, sums columns, and places valid records.

The same sorting schedule uses either the NumPy difference channel or SdkDifference. In the SDK path, every matrix entry is actually evaluated through linear_fprop in the official CPU backend; a noiseless check of all saved lists confirms stable output. A batched API call does not establish simultaneous physical execution of every entry, optical summation, or optical routing. [D2]

Periodic scores cannot replace D unchanged: the bias Δ is defined in difference units. A nonlinear implementation requires a separate derivation of thresholds and data flow. It therefore remains a separate pair-level feasibility study in Chapter 7.

6.7 Noise and threshold margin

The main evaluation adds a separate assumed Gaussian error to every matrix entry, including the diagonal. Neither reverse comparisons nor self-comparisons are corrected afterwards. Shared inputs could cause correlated errors on actual hardware; the independent model is an explicitly assumed sensitivity study.

The raw source model contains zero threshold margins. For [1,0], the lower-triangular difference is Δ and becomes exactly zero after the bias. A small negative error removes that contribution. Rounding to grid Δ beforehand creates a half-step margin to the decision boundary. Four controls with 100,000

pairs each test the resulting Gaussian probabilities. Only these isolated pair checks hold the diagonal exact; the main evaluation measures and perturbs it. [D8]

The auxiliary matrix resolves ties under exact arithmetic, but not inconsistent noisy comparisons. This equation check does not fully reproduce physical smart-pixel noise, optical rank summation, bias errors, or real channel dependencies.

7. Q.ANT SDK and periodic comparison

7.1 Verified software version

The inspected SDK is version 2.3.1, source commit 72a2d99f10240b6df3c6d0f636dfa0e2b5d38902. This chapter's statements apply to that version; a later SDK would require a separate audit. The CPU release and its manufacturer licences are included in the repository. [36]

Table 10. Verified SDK functions and their limits

Function	Path visible in the inspected source	Limit of the evidence
linear_fprop	Matrix/vector driver call	Documented API path; no measured hardware performance
calc_scaled_periodic_nl_fprop	Native periodic driver function v·tcos(u)	CPU substitute does not establish the physical transfer characteristic or resolution
relu_fprop	Host loop using max(x,0)	No evidence of native optical ReLU
KAN composition	Driver calls plus host work	Hybrid execution; no established purely optical cascade
Complete C&E	No corresponding verified primitive path	Optical routing and cascading remain unresolved

Our illustration based on the derivation or sources cited in this section.

The CPU backend substitutes cosine for the periodic function. It is not a calibrated digital twin of the optical chip. The closed native driver also limits visibility into internal buffers, conversions, and scheduling.

7.2 Deriving the candidate

For normalized differences $-1 < d < 1$, phase u is mapped onto the monotone cosine branch between 0 and π:

$$u = u_0 - \alpha d, \quad u_0 = \alpha = \frac{\pi}{2}, \quad \cos(u) = \sin(\alpha d)$$

The ideal score therefore has the same sign as d. This construction provides an ordering signal, not exact minimum/maximum values. A physical function f would need a suitable monotone transfer characteristic with sufficient resolution over the selected range. Periodicity alone is insufficient.

With finite CPU precision, a reference $y_0 = f_CPU(d = 0)$ is subtracted. BF16 rounds π/2 to 1.5703125. The saved CPU result for the zero reference is 0.000484466552734375. Without correction, equal pairs would produce a positive signal. With $z = y - y_0$, all 256 ordered 4-bit pairs and 65,536 ordered 8-bit pairs in the saved check are correct, including ties. [D3]

Table 11. Exhaustive periodic pair check in the official CPU backend

Keys	Ordered pairs	Errors before reference	Errors after reference	Minimum margin m
4 bits	256	16	0	0.10155869
8 bits	65,536	256	0	0.00780869

Source: results/periodic/pair_checks.json. No added noise; no hardware execution.

For the 8-bit pair (10,9), the exact normalized difference is 1/256. After host phase calculation and BF16 rounding, u = 1.5625; the periodic CPU output is 0.00830078125. Reference subtraction gives z = 0.007816314697265625 > 0. The equal pair (9,9) produces z = 0. Discrete BF16 phase steps explain why the smallest saved score gap differs from the smooth ideal model.

7.3 What the component actually contributes today

In the existing execution path, d already resides on the host after the linear API call. There, sign(d) and the tie rule can immediately provide the comparison decision. The subsequent periodic call adds phase calculation, another call, buffers, and final thresholding. Its functionality is established; a benefit over the direct decision has not been demonstrated in this path.

The required nonlinearity is the ordering decision. Even after the periodic transformation, the electronics interpret the sign or a tolerance band. The additional call does not avoid an electronic intermediate step in the current code.

7.4 Affine phase calculation: feasibility and precision limit

The host phase calculation can be moved algebraically into the linear operation. For normalized keys $x = a/2^w$ and $y = b/2^w$ with key width w:

$$u = [x, y, 1] \cdot [-\alpha, \alpha, u_0]^\mathrm{I}, \quad u_0 = \alpha = \pi/2$$

The constant input one supplies the bias. check_affine_periodic.py executes exactly this BF16 MVM followed by the periodic SDK call. It does not calculate a difference on the host. However, the phase is still returned as a NumPy array and passed to the second call. This API composition removes host phase arithmetic but saves neither a call nor the visible intermediate buffer. No documented resident MVM/nonlinearity path was found in the inspected public contract. This does not exclude further capabilities in the closed hardware driver. [36, D7]

Algebraic equivalence does not automatically preserve finite-precision behavior. The CPU backend rounds every MVM product to BF16, sums in FP32, and rounds the output to BF16 again. With separate products αx and αy, information about a small key difference may be lost before summation. The additional checks use no noise, further fixed-point rounding, or tolerance band. [36, D7]

Table 12. Affine CPU phase compared with two controls

Width	Pairs per path	Direct: errors	Host phase: errors	Affine: errors
4 bits	256	0	0	0
8 bits	65,536	0	0	40

Source D7: summary.json; all ordered pairs, including ties. SDK 2.3.1, no added noise.

For the 8-bit pair (165,166), the three rounded products are −1.015625, +1.015625, and 1.5703125. This produces the same phase as a tie. After subtracting the single reference from (0,0), the score is zero. All 40 errors are false ties; there are no sign reversals or broken true ties. The smallest score magnitude over all unequal 8-bit pairs is therefore zero. The smallest nonzero score must not be presented as an available minimum margin. Reference correction and a tolerance band cannot restore lost information.

This investigation is limited to the comparison function. Its scores are not used as the difference matrix of the canonical Louri sorter.

7.5 Consequence for the feasibility study

The tested affine variant is not adopted as a correct 8-bit comparator. The negative result is limited to this encoding and CPU arithmetic; it does not exclude other encodings or different physical arithmetic precision. Before further optimization, bias representation, internal precision, input ranges, and an actually supported data path must be clarified. The existing host-phase variant remains a correctly functioning control.

For P pairs, logical BF16 buffer volumes without calibration or assumed caching are 6P + 4 bytes for the direct path, 12P + 4 bytes with host phase calculation, and 14P + 6 bytes for the affine variant. The latter requires a third input column. These counts cover public input/output buffers and weights, not measured PCIe transfers. Moving the arithmetic therefore provides no established resource saving in the inspected API path.

A useful internal connection would, for example, need to avoid an intermediate conversion while demonstrating sufficient accuracy and lower overall cost. Execution without a host round trip would still not establish an unregenerated optical cascade. The public KAN composition itself contains explicit CPU phase addition between driver calls. A higher-level SDK function alone therefore does not demonstrate such a cascade. [36]

The periodic variant remains a limited feasibility study. Its role is now bounded by a reproducible precision finding and specific manufacturer questions. The existence of a native nonlinear function establishes neither its suitability for this subproblem nor a system advantage.

7.6 Manufacturer claims and unresolved device parameters

The publicly checked Q.ANT product page lists PCIe Gen4 x8, 8 GOPS, an NPU power figure of 150 W, and a CPU simulation path. These specifications describe neither a sorting result nor a guaranteed comparison rate. The number of processing channels does not define a square MVM dimension. Actual transfer costs and the smallest reliably distinguishable signal must be determined for the specific kernel. [37, accessed 7 October 2026]

The tcos transfer characteristic, input ranges, resolution, drift, and noise remain unresolved. A measurement runner and inquiry draft are prepared; physical measurements and a manufacturer response are not documented. Chapter 15 describes the required data.

8. Reference experiment: inputs, quantization, and noise

8.1 Corpus and experimental space

The saved key ranges are 0 to 15 and 0 to 255. For 4 bits, N = 2, 4, 8, 16; for 8 bits, sizes additionally include 32, 64, 128, and 256. Each combination of width, N, and input family contains 1,000 lists. Two families are distinguished: distinct keys, and uniform sampling with replacement, which permits ties. The corpus contains 24,000 lists in 24 datasets.

The “duplicates allowed” family does not guarantee a tie in every small list. It describes the generation process. In the maximum distinct case $N = 2^b$, each list contains the entire key domain in a different order.

Inputs are validated before processing. Integral numeric values such as 3.0 are allowed; booleans, strings, fractional values, and values outside the declared range are rejected with their position. The saved corpus is reused unchanged. New noise conditions are not new independent input corpora.

8.2 Widths and normalization

b_key determines the valid key range, b_in the input encoding, and b_out the modelled difference output. Current reference runners support the agreed matched triples (4,4,4) and (8,8,8). Unmatched triples are conceivable separate parameters but are not an evaluated experimental space.

With largest valid key M and number of input levels L_in:

$$M = 2^{b_{\text{key}}} - 1, \qquad L_{\text{in}} = 2^{b_{\text{in}}}$$

The declared scaling is:

$$u(x) = \frac{x}{M}\left(1 - \frac{1}{L_{\text{in}}}\right), \quad \widehat{x} = \Delta_{\text{in}}\,\text{roundEven}\left(\frac{u(x)}{\Delta_{\text{in}}}\right)$$

With $\Delta_{in} = 1/L_in$ and matched key/input widths, $\hat{x} = x/2^b$ exactly. Scaling uses the complete declared range, not the minimum and maximum of an individual list. Signal spacing therefore remains comparable across lists.

8.3 Difference channel and subsequent sorting decision

$\Delta_{in} = \Delta_{out} = \Delta = 2^{-b}$. The comparator forms $d = \hat{a} - \hat{b}$, adds Gaussian ε with standard deviation $\sigma = \eta\Delta$, rounds to the nearest even integer code, and saturates to the b-bit code range:

$$c = \text{roundEven}\left(\frac{d+\varepsilon}{\Delta}\right), \quad c_{\text{lim}} = \text{clip}\left(c, -2^{b-1}, 2^{b-1}-1\right), \quad \widehat{d} = c_{\text{lim}}\Delta$$

The six η values are 0, 0.05, 0.10, 0.15, 0.20, and 0.25. They are synthetic sensitivity values in units of the difference grid. They are neither measured Q.ANT disturbances nor error probabilities or percentages of optical power.

Without added noise, $d/\Delta = a - b$ is an integer. Rounding does not change that code; saturation changes neither its sign nor its zero status. The Bitonic sign decision is therefore exact. For Louri, every positive code also remains at least one grid step, so it is still nonnegative after the lower-triangular bias. The noiseless matrix decisions therefore remain exact too. This statement does not apply to reconstructing the difference magnitude.

8.4 Why the same measurement error has different consequences

For (10,9) at 8 bits, d = 1/256. An error of −3/1024 reduces (d+ε)/Δ to 0.25, which rounds to zero. Bitonic treats this signal as a tie using original indices. In the Louri matrix, each measured value is first shifted by its position-dependent bias and then thresholded separately. The two directions can therefore produce inconsistent contributions.

A self-comparison is also a measurement event. Starting from d = 0, $\varepsilon < -\Delta/2$ can produce a negative rounded value; the expected diagonal rank contribution is then missing. In long lists, these errors can occur alongside incorrect pair decisions. The implementation does not replace this measurement with the known exact answer.

The main evaluation explicitly uses $\Delta_{in} = \Delta_{out}$. Noise, quantization, and bias location are documented separately in the metadata; there is no hidden second rounding after the electronic bias.

8.5 Execution and statistics

Each width-size-family combination is evaluated at six noise strengths with two mappings: 24 × 6 × 2 = 288 configurations, each containing 1,000 lists. Inputs, seeds, model parameters, and output files are saved. Batch size affects runner memory; absolute trial identifiers ensure that it does not change the random experiment.

The complete correctness rate covers all trials, including invalid rank outputs. Wilson intervals describe the binary success rate per configuration. A result of 1,000 successes out of 1,000 corresponds to an approximately 95% interval from 99.62% to 100%; it does not prove universal correctness for arbitrarily many future inputs. Shared inputs and control streams must not be counted as independent repetitions.

9. A shared signal and noise model

9.1 Why signal spacing and disturbance location must be defined together

Noise on the difference before rounding is not the same experiment as the same numerical amplitude on a nonlinear output without rounding. The transfer function changes signal spacing; rounding changes decision regions. A quality difference between these scenarios may therefore arise from scaling and quantization and does not alone establish a physical advantage.

The shared model separates these processing steps in explained units. Direct and ideal periodic transfer functions are compared in pair checks and Bitonic controls. The complete Louri sorter uses only the direct difference channel because its bias is defined in those units.

9.2 Common coordinates and processing steps

The control model uses normalized difference units and an explicit sequence. β_{in} and β_{out} are fixed residual offsets; ε_g is a relative gain error. Z_in and Z_out are independent standard normal variables per comparison. Q_q rounds with step q to the nearest even code; Q_0 is the identity.

$$z = \mathrm{clip}_{[-1,1]}\left(Q_{q_{\mathrm{pre}}}(d + \beta_{\mathrm{in}} + \sigma_{\mathrm{in}} Z_{\mathrm{in}})\right)$$

$$y = \mathrm{clip}_{[-1,1]}\left(Q_{q_{\mathrm{out}}}((1+\varepsilon_g) g(z) + \beta_{\mathrm{out}} + \sigma_{\mathrm{out}} Z_{\mathrm{out}})\right)$$

The two ideal transfer functions are:

$$g_{\mathrm{direct}}(d) = d, \qquad g_{\mathrm{periodic}}(d) = \frac{\sin(\alpha d)}{\alpha}, \qquad \alpha = \frac{\pi}{2}$$

Both have unit slope at the decision point. If a physical output scale is divided by α, its disturbances must also be divided. Normalization does not create a free precision improvement. The periodic curve becomes flatter near the range boundary; equal slopes near zero do not make the complete transfer functions identical.

The control model executes in Float64. It is neither the BF16 SDK nor a digital twin of Q.ANT hardware. Clipping to [−1,1] enforces the monotone region considered here. A symmetric quantizer with step Δ over this entire range contains more than 2^b codes and is therefore not described as a b-bit ADC.

Table 13. Parameters and their evidence status

Quantity	Saved control	Rationale / limitation
Key grid Δ	2^{-4} or 2^{-8}	Integer-corpus normalization
σ_{in} / Δ; σ_{out} / Δ	0 or 0.25	Assumed sensitivity, not a measurement
q_pre	0 or Δ	No or shared intermediate rounding
q_out	0; 0.5Δ; Δ; 2Δ	Separate resolution assumption in pair cases
β_{out}	−0.25Δ; 0; +0.25Δ	Fixed residual offset, no drift trajectory
β_{in}; ε_g	0; 0	Supported effects disabled here
Transfer function and range	d or sin(πd/2)/(π/2); [−1,1]	Ideal Float64 functions, not a tcos fit
Noise distribution	Independent Gaussian noise	Heavy tails and correlations remain unresolved

Source: results/common_noise/metadata.json and docs/COMMON_NOISE_MODEL.md. Not every combination is tested as a complete sort.

The saved sorting controls use selected parameter combinations. The broader pair checks also vary residual offset and output grid. Input offset and gain error are zero in these saved controls. Fixed offsets are not time-dependent drift; no drift trajectory has yet been measured or simulated.

9.3 Incorporating supported physical values later

López-March et al. analyze APC noise and motivate separating physical sources, their transfer, and their correlations. One example specifies 1,550 nm, a laser linewidth of 400 kHz, and RIN of −145 dB/Hz. These values belong to that setup. They provide neither receiver bandwidth nor a Q.ANT comparator variance and are not used here as Q.ANT simulation parameters. [21, p. 8 of the supplied version]

For an idealized receiver with flat spectral densities, current-noise variance can, for example, be expressed as:

$$\sigma_I^2 = B_{\mathrm{eq}}\left(S_{\mathrm{th}} + 2q_e I + 10^{\mathrm{RIN_{dB}}/10} I^2\right)$$

I is the positive photocurrent in one detector branch, not simply a signed difference. B_eq is equivalent noise bandwidth and q_e the elementary charge. Differential readout, multiple channels, and correlated sources require propagation of the full covariance through the calibrated transfer. In the linearized case this is $J\Sigma J^{\mathsf{T}}$. Laser linewidth must not be substituted for B_eq.

For the simplified scalar transfer function with independent input and output disturbances, locally:

$$\mathrm{Var}(y \mid d) \approx (1+\varepsilon_g)^2 [g'(d)]^2 \sigma_{\mathrm{in}}^2 + \sigma_{\mathrm{out}}^2$$

Quantization, clipping, and nonlinearity can limit this linear approximation. Reference subtraction additionally requires Var(y − y_ref) = Var(y) + Var(y_ref) − 2 Cov(y,y_ref). A reference error measured once is shared across many comparisons. Redrawing it independently for each comparison would imply a different hardware assumption.

9.4 Controls before large sorting runs

The control series tests 280 pair cases with 100,000 samples each, totalling 28 million decisions. For 182 suitable cases, analytical probabilities are checked against Monte Carlo results. The tolerance includes six standard errors and a small finite-sample allowance; it is an implementation check, not a claim about Q.ANT.

With shared intermediate quantization, 1.4 million paired direct and ideal-periodic decisions are identical. The sorting control covers 108 configurations and 10,800 sorts: six datasets, six settings, and three admissible mapping/transfer combinations, using the first 100 saved lists in each case. Bitonic uses direct and ideal-periodic values; Louri uses only direct differences followed by bias and threshold. All 18 noiseless configurations are correct and stable. The 600 matched Bitonic outputs with pre-quantization are identical. The periodic transfer function is not combined with Louri's difference bias without a separate derivation.

10. Quality metrics: complete order and useful ranking

10.1 Four different questions

An output is **valid** if it contains every original record exactly once. It is **correctly sorted** if its keys are monotone. It is **stable** if equal keys retain their original order. A partially incorrect output can still provide a very good approximate ranking. These properties are evaluated separately; invalid outputs are not repaired into permutations.

Complete correctness remains necessary for exact sorting. However, it does not reveal whether one adjacent pair or large portions of the list are out of order. Kendall-τ, inversions by key distance, and Recall@k therefore supplement the binary success rate.

10.2 Kendall-τ-b with equal keys

For a valid output, positions are compared with the corresponding original keys. There are $C = N(N-1)/2$ position pairs. M have unequal keys; I of these M pairs are inverted. Positions have no ties, whereas keys may have ties, so:

$$\tau_b = \frac{M - 2I}{\sqrt{CM}}, \qquad M > 0$$

If every key is distinct, M = C and perfect order has $\tau_b = 1$. With duplicates, the attainable ceiling is $\sqrt{(M/C)}$, even if no unequal pair is inverted. This follows from tie correction, not a sorting error. The report therefore also shows the ceiling and, where helpful, the separately named quantity $\tau_{key} = 1 - 2I/M$. It normalizes only unequal-key pairs and must not be presented as standard Kendall-τ-b. [39]

For all-equal keys, M = 0 and the metric is undefined. Invalid rank outputs also have no τ in this evaluation. Means over valid outputs are always reported with their count; they must not conceal a high invalid-output rate.

10.3 Errors by key distance

For each unordered pair of distinct original keys, the integer distance $g = |a - b|$ is computed. For each g, the evaluation counts inverted and total comparable pairs in valid outputs. Their ratio is a conditional inversion rate. It reveals whether the output mainly confuses nearby keys.

This metric does not reconstruct internal comparator errors. A disturbed comparison may later be corrected or trigger other data movements. The final permutation reveals its order and inversions, not every preceding decision. Invalid rank cases contribute to the separate validity rate and have no pair denominator here.

10.4 Recall@k and fair handling of the selection boundary

The intended output order is ascending, so top-k selection uses the last k positions. These are compared with the largest k elements of the stable digital reference. Saved k values are 1, 5, 10, and 32 where $k \leq N$, plus N as a full-set control.

The ID-based variant counts the intersection of selected original indices with the stable reference and divides by k. A tie at the selection boundary can lower this metric if different, equally suitable records are chosen. The tie-neutral variant therefore credits all selected keys above the threshold and permits exactly the remaining required number of threshold-valued records. It never exceeds one.

Both the mean over valid outputs and a mean over all trials, assigning zero to invalid outputs, are reported. Recall@N is one for every valid permutation and checks only set preservation. It does not replace an ordering metric.

10.5 Relation to PRISM and limits of comparison

PRISM investigates selection of relevant KV-cache blocks for long LLM contexts through photonic similarity scoring. A sufficiently good selection may suffice even without an exact total order. This motivates the top-k perspective. Our sorting evaluation nevertheless uses different data and a different application criterion; its recall is not a PRISM benchmark and does not directly establish LLM quality. A later application comparison needs matching scores, selection sizes, and quality targets. [23]

10.6 Complete current outputs

All 288 current reference configurations, totalling 288,000 outputs, are saved. Quality evaluation rereads these indices, reconstructs original keys, and checks validity, correctness, and stability against the saved counts. No configurations are missing. It does not rerun a sorter. [D1, D4]

Matrix rank sorting can produce invalid outputs. Kendall-τ, key-distance errors, and conditional recall are undefined for these. Tables therefore identify the valid subset and additionally report an explicitly named recall variant assigning zero to invalid outputs.

11. Results from saved experiments

11.1 Scope and levels of evidence

The current experiment series use Bitonic and exclusively the complete Louri matrix. The numerical reference, SDK integration, pair feasibility, shared noise controls, and digital timing answer different questions; their run counts must not be combined as independent hardware measurements.

Table 14. Current experiment series

Series	Scope	Purpose
Bitonic / full rank matrix	288 configurations; 288,000 sorts	All outputs saved
Linear CPU SDK	48,000 noiseless sorts	N^2 rank cells through the official API
Louri source equations	155,328 inputs; 400,000 boundary-case pairs	Stable ranks and analytical noise controls
Periodic pairs	256 + 65,536 pairs	Finite-domain CPU check
Affine CPU phase	3 × (256 + 65,536) pairs	Precision limit against two controls
Ranking metrics	288,000 existing outputs	Complete coverage; no new sorting run
Shared model	28 million pair decisions; 10,800 sorts	Defined noise locations, scales, and quantization
Digital CPU baseline	192 configurations; 1,728 repetitions	Stable reference with batch/type variants

11.2 Complete sorting correctness and rank placement

Without added noise, both methods are correct and stable on all saved inputs. Under the assumed independent errors, N^2 rank contributions, including the diagonal, become vulnerable. A small per-measurement error rate can invalidate many complete outputs for long lists. [D1]

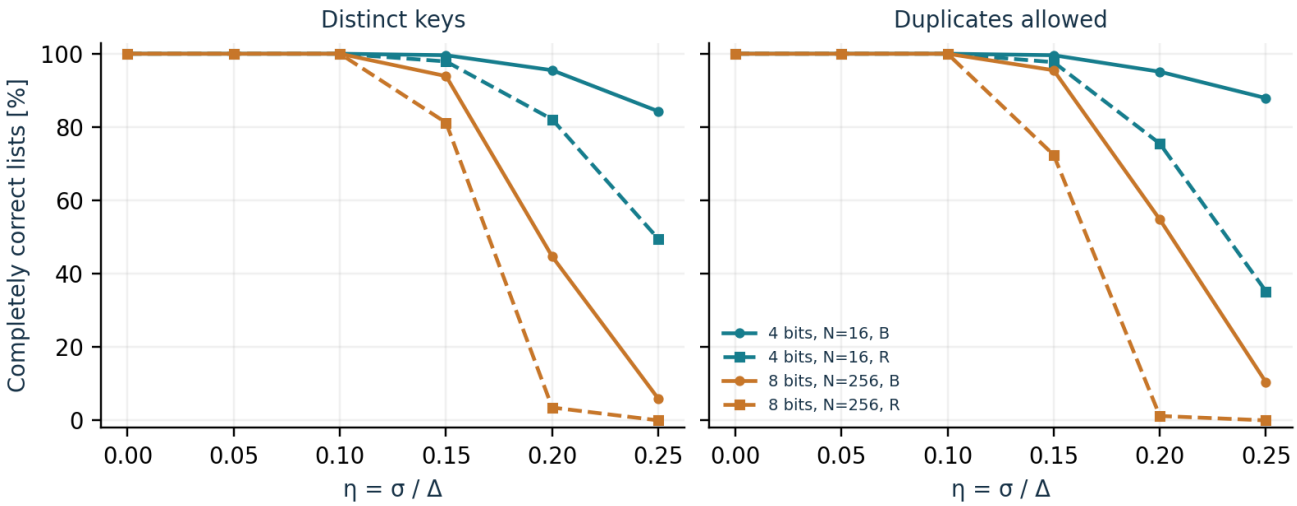


Figure 1. Reference model at the largest tested list size. B: Bitonic, R: rank sorting; 1,000 lists per point. Source: results/reference/accuracy.csv.

Table 15. Complete correctness at the largest tested list size

Width / N	Inputs	η	Bitonic correct	Rank correct
4 bits / 16	distinct	0.15	99.6 %	97.9 %
4 bits / 16	distinct	0.20	95.5 %	82.1 %
4 bits / 16	distinct	0.25	84.3 %	49.5 %
4 bits / 16	Duplicates allowed	0.15	99.6 %	97.7 %
4 bits / 16	Duplicates allowed	0.20	95.1 %	75.5 %
4 bits / 16	Duplicates allowed	0.25	87.9 %	35.2 %
8 bits / 256	distinct	0.15	93.9 %	81.2 %
8 bits / 256	distinct	0.20	44.7 %	3.5 %
8 bits / 256	distinct	0.25	5.9 %	0.0 %
8 bits / 256	Duplicates allowed	0.15	95.5 %	72.4 %
8 bits / 256	Duplicates allowed	0.20	54.8 %	1.2 %
8 bits / 256	Duplicates allowed	0.25	10.4 %	0.0 %

Source: results/reference/accuracy.csv; 1,000 trials per cell. Invalid outputs count as failures.

At 4 bits, N = 16, and η = 0.25 with distinct keys, Louri produces 500 valid, 495 correct, and 495 stable outputs from 1,000 lists. With duplicates allowed, the counts are 354 valid, 352 correct, and 350 stable. At 8 bits and N = 256 with this noise strength, no matrix output is valid in either family. This is a result of the stated independent measurement model, not a measured hardware failure.

11.3 Ranking quality must accompany validity

Table 16. 8 bits, N = 256, distinct keys, η = 0.25

Metric	Bitonic	Rank sorting
Valid outputs / 1,000	1000	0
Completely correct	5.9 %	0.0 %
Mean τ_b, valid outputs	0.999826	–
Mean inversions, valid outputs	2.847	–
Recall@10, valid outputs	99.880 %	–
Recall@10, invalid outputs = 0	99.880 %	0.000 %

Sources: results/ranking_quality/summary.csv and recall.csv. Both recall variants agree for distinct keys.

Bitonic preserves a complete permutation even with incorrect decisions. At 8 bits, N = 256, and η = 0.25, low complete correctness can therefore coexist with very high Kendall-τ. Louri has no valid outputs under this condition, so conditional τ and conditional recall are shown as undefined. High quality in a small valid subset must not be presented as the quality of all trials.

11.4 Nonlinear pair feasibility

The periodic host-phase path correctly decides all investigated pairs after reference correction. The affine variant instead produces the 40 false 8-bit ties explained in Chapter 7. These pair findings remain relevant to a possible nonlinear implementation; they are not presented as complete Louri sorting results. [D3, D7]

11.5 Shared model: specified disturbance locations and quantization

Table 17. Shared model: selected complete sorts

Setting	Mapping	Valid / 100	Correct / 100	τ_b, valid	Recall@10, all
Shared intermediate rounding	Bitonic	100	8	0.999814	99.8 %
Shared intermediate rounding	Rank	0	0	–	0.0 %
Input and output noise	Bitonic	100	0	0.999371	99.3 %
Input and output noise	Rank	0	0	–	0.0 %
Coarse output grid 2Δ	Bitonic	100	0	0.996376	97.3 %
Coarse output grid 2Δ	Rank	0	0	–	0.0 %

Source: results/common_noise/sort_quality.csv. 8 bits, N = 256, distinct keys. The direct difference channel is shown; the rank variant uses the full matrix including its diagonal. “All” assigns zero to invalid outputs.

The control series contains 28 million pair decisions and 10,800 complete sorting evaluations. Pair cases test direct and ideal periodic transfer functions. The 108 sorting configurations combine two Bitonic transfer functions with one direct Louri channel. The direct rank channel, quantized before the bias, is thereby clearly distinguished from a hypothetical nonlinear rank implementation. All 18 noiseless sorting configurations are correct and stable. [D5]

11.6 Digital CPU baseline

The digital baseline sorts the same 24,000 saved inputs using NumPy argsort(kind="stable") and gathers the corresponding keys. It returns both sorted keys and original indices, meeting the same stable-output contract. All inputs were checked for correctness and stability outside the timed region. [38]

The measurements cover 192 configurations: 24 datasets × two storage formats × four batch sizes. Each has nine timing repetitions after two warm-up passes and calibration to approximately 15 ms measurement duration. Configuration order was reproducibly shuffled. Input-file reading, type conversion, and validation are excluded; the Python call, output-array allocation, sorting, and gathering are included. The experiment represents a warm execution path, not a cold start from a file.

The environment reports an AMD EPYC 9V74, Python 3.12.14, and NumPy 2.3.5 on Linux x86-64. Nine logical CPUs are visible; the cgroup quota is eight CPU equivalents. This does not imply that the sorting function uses eight threads. Processor affinity was not set.

Table 18. Digital baseline: 8 bits, N = 256, distinct keys

Type	Lists per batch	Batch time [µs]	Amortized [µs/list]	Rel. IQR
int64	1	11.90	11.90	5.4 %
int64	1,000	8,851.66	8.85	12.2 %
uint8	1	5.26	5.26	10.6 %
uint8	1,000	1,407.79	1.41	3.7 %

Source: results/cpu_baseline_20261007/summary.csv. Median of nine repetitions; see text for CPU and software environment. Not an NPU comparison.

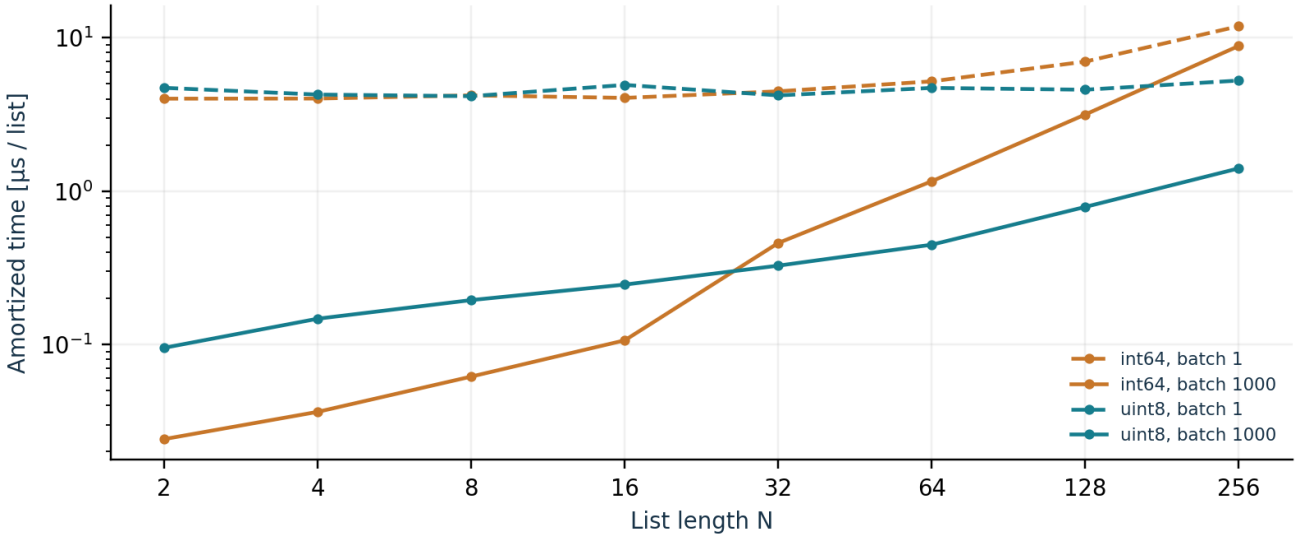


Figure 2. CPU baseline for 8-bit inputs with distinct keys. Amortized batch times, not individual request latencies. Source: results/cpu_baseline_20261007/summary.csv.

The reported value is the median of nine measured mean batch times. Amortized time per list is batch time divided by batch size, not an individually measured response time. The interquartile range describes variation across these repetitions; it is neither a confidence interval nor a distribution of individual request latencies. Four of 192 configurations have a relative interquartile range above 20%; they were not removed.

Differences between uint8 and int64 show that storage format and sorting path matter. This baseline is reproducible and meets the output contract, but does not claim to be the fastest possible CPU implementation. Without a measured complete NPU run, no photonic speedup factor can be derived. A GPU baseline and energy recording remain absent.

12. Calls, data movement, and potential system costs

12.1 Logical accounting without invented device capacity

The following counts describe the intended SDK mapping. They count payload data and calls, not measured PCIe transactions. For a capacity c of linear pair operations per call, stagewise Bitonic execution gives:

$$q_B = K\left\lceil \frac{N/2}{c} \right\rceil, \qquad q_R = \left\lceil \frac{C_R}{c} \right\rceil$$

c is a capacity assumption, not a publicly confirmed Q.ANT limit. BF16 inputs logically require four bytes per pair and difference output two bytes. Resubmitting the two weights adds four bytes per linear call. Buffer management, driver protocol, and internally reused data are excluded.

Table 19. Direct linear mapping when each stage or rank batch fits in one call

N	Mapping	Comparisons	Linear calls	Logical bytes
16	Bitonic	80	10	520
16	Louri	256	1	1,540
64	Bitonic	672	21	4,116
64	Louri	4,096	1	24,580
256	Bitonic	4,608	36	27,792
256	Louri	65,536	1	393,220

Our accounting: 4 input bytes + 2 output bytes per BF16 pair; 4 weight bytes per call. Not measured driver or PCIe bytes.

For N = 256 and a sufficiently large rank batch, 36 linear Bitonic calls contrast with one rank call. Louri requires 65,536 comparisons and 393,220 logical bytes; Bitonic requires 4,608 comparisons and 27,792 bytes. The rank variant therefore needs approximately 14.1 times as many logical bytes. At c = 128, it needs 512 rank calls versus 36 Bitonic calls. The apparent single-call advantage depends strongly on actual capacity.

12.2 Role of electronic processing

In the canonical SDK mapping, the linear operation returns a difference to the host. Louri then requires bias subtraction, thresholding, N column sums, and placement of original records. Bitonic needs a decision and exchange after each stage. This work and pair packing must be included in any future complete runtime measurement.

A subsequent periodic call is unnecessary when the difference decision is already electronically available. Its possible role depends on a demonstrated internal path that performs useful additional work or avoids a transfer. A connection without a host round trip would still not prove entirely optical processing.

12.3 A conditional runtime equation

For completely serial execution without overlap, the physical time of architecture A can be decomposed in a model as:

$$t_A = q_A \ell + C_A t_{\text{arith}} + \frac{U_A}{B_{\text{up}}} + \frac{V_A}{B_{\text{down}}} + h_A$$

ℓ is fixed call overhead; U and V are transferred payload bytes; B_up and B_down are effective bandwidths; h_A is host work, including packing, decisions, ranks, and data movement. These auxiliary quantities describe a future physical measurement and do **not** extend complexity vector C. Concurrent transfers, caching, and actual parallel processing require an adapted execution model.

Under the same linear-payload assumptions, rank sorting would outperform Bitonic only if saved fixed call overhead outweighed its additional comparisons, payload bytes, and host costs. Fewer API calls alone do not imply lower total runtime. Likewise, a PCIe link's datasheet peak bandwidth is not measured effective application bandwidth.

12.4 Energy requires a clear measurement boundary

The literature explains why optical operations and complete systems must be evaluated differently. Lasers, losses, modulation, ADC/DAC, memory, the host, and cooling can determine energy use. A component-level figure or high internal parallelism cannot be directly converted into joules per completely sorted list. [1, 22, 33]

A future energy baseline must satisfy the same inputs, outputs, and quality requirements. Records must include the measurement boundary, active and, where applicable, idle power, time integration, batch size, and uncertainty. Any subtraction of idle energy must be explicitly specified, with total energy reported as well. The cited manufacturer figures of 8 GOPS and 150 W are specifications, not measurements of this sorting workload. [37]

13. From concept to executable code

13.1 The current execution chain

input_validation.py checks keys. comparison.py implements the explained normalized difference channel. sorting_schedules.py contains the active Bitonic layers and complete Louri procedure. louri_reference.py provides the source equations with inspectable intermediate matrices. metrics.py and ranking_quality.py independently evaluate completed outputs.

Table 20. Code overview by task

Files under qant_sorting/	Function and limitation
input_validation.py; comparison.py	Integer domain and normalized difference channel
sorting_schedules.py; louri_reference.py	Bitonic and full Louri matrix; inspectable source equations
metrics.py; ranking_quality.py	Validity, correctness, stability, τ, recall, and distance errors
sdk_mapping.py; periodic_comparison.py	Linear SDK integration and separate periodic pair function
common_noise_model.py	Explained signal stages, quantization, and disturbance parameters
experiments/	Explicit experiment, verification, resource, and measurement commands
report.py; report_assets/thesis_report.md	Current complete report from sources and hash-verified evidence
tests/	Independent mathematical, API, and reproduction checks

13.2 Data contracts and error handling

Inputs are rectangular batches with shape (trials,N). Louri's pair shape is (trials,N^2); both directions and the diagonal are submitted. The bias_step parameter must explicitly match the difference grid. Invalid rank placement returns −1 markers and is not repaired. Raw keys use signed integers so differences and error markers do not wrap.

13.3 One current version

The repository contains one rank implementation for the thesis and its current results. Obsolete prototypes, duplicate input archives, unused sorting runs, recovery tools for old archives, and earlier report copies are absent from the current file tree. Version comparisons remain available through Git. The supplied SDK archive is a required, licensed dependency; result NPZ files contain the original outputs needed for independent rescoring.

14. Reproducibility and verification

14.1 Identity of inputs and SDK

The SHA-256 of the unchanged input corpus data/inputs.npz is:

79efbd1ee270242cc122d8f9e1848fe07a85077bcf310324b27dd60f8faf11da

CPU SDK verification uses version 2.3.1 and the pinned source commit:

72a2d99f10240b6df3c6d0f636dfa0e2b5d38902

SDK source audits are stored in docs/sdk_source_audit.json and docs/nonlinearity_source_audit.json. Each experiment series records input identity, model assumptions, seeds, and the source hashes present at execution time. report_assets/evidence.json binds the result files actually used to the report; sources.json documents the supplied literature.

14.2 Environments and limitations

Numerical evaluations use Python 3.12 and NumPy 2.3.5; requirements/analysis.txt also contains SciPy 1.17.0 for independent metric checks. requirements/sdk.txt adds ml-dtypes 0.6.0, cffi 2.1.1, and pycparser 3.0. The official CPU wheel is installed from the hash-verified vendor archive. Report rendering uses separate dependencies in requirements/report.txt.

Digital timing remains tied to its recorded CPU, operating system, and software environment. A rerun may produce different times. Numerical and SDK results are not photonic timing measurements.

14.3 Reproduction commands

```
python -m pip install -r requirements/analysis.txt
python -m unittest discover -s tests -v
python -m qant_sorting noise
python -m qant_sorting verify results/reproduced
python -m qant_sorting louri
python -m qant_sorting quality
python -m qant_sorting common-noise
python -m qant_sorting resources
```

In an SDK environment:

```
python -m pip install -r requirements/sdk.txt
python -m qant_sorting install-sdk --no-deps
python -m qant_sorting sdk
python -m qant_sorting affine
python -m qant_sorting periodic-pairs
```

New runs write to separate reproduction directories. The default quality evaluation reads the saved current reference. python -m qant_sorting report builds the single current PDF in reports.

14.4 What the checks establish

Checks cover the published matrices, stable ties, rank numbering, negative or duplicate ranks, all N^2 observations, complete SDK pair submission, reproducible random streams, and independent quality scores. Deliberately injected inconsistent reverse decisions must not be corrected by complementing one direction. Noise-event order is specified for every matrix and absolute trial index.

These checks do not establish a physical transfer characteristic, optical rank summation, physical cascading, a device-specific error distribution, or energy efficiency.

15. Hardware questions and outstanding work

15.1 What is publicly established and what remains unknown

Public evidence covers the inspected SDK contract, distinct CPU/hardware connections, and manufacturer product claims. These do not establish actual allowable phase and amplitude ranges, comparator resolution, drift, channel dependence, the physical tcos characteristic, or sorting time. References to systems at computing centres do not prove external access available to this thesis. [36, 37]

An inquiry template and measurement plan are available. The draft also asks about a resident affine MVM/nonlinearity connection and whether CPU-backend BF16 product rounding matches actual device precision. The evaluated project state documents neither a sent inquiry, a manufacturer reply, nor usable NPU access. No manufacturer commitment or physical measurement is therefore claimed.

Table 21. Required manufacturer information and measurements

Question	Specific data collection	Value for the mapping
Allowable u/v ranges	Versioned driver contract, units, and channel normalization	Allowed phases and amplitudes
Transfer characteristic and zero point	Dense sweeps plus the difference codes used	u_0, monotone interval, and minimum margin
Resolution and rounding	Code histograms, DAC/ADC settings	Quantization distinguished from BF16
Noise and correlation	Repetitions per level/channel; raw distributions and tails	Physical parameters replacing the assumed η
Drift and calibration	Time-stamped references, load/temperature sequences	Shared errors and calibration interval
Capacity and cascade	Confirmed dimensions, buffer residency, and data path	Call count and avoided host steps
Time and energy	Cold/warm runs, transfers, complete power traces	Same task compared with CPU/GPU

Basis: docs/QANT_MEASUREMENT_PLAN.md. Inquiry template available; no documented measurement access.

15.2 Prepared transfer-characteristic sweep

measure_periodic_curve.py can record phase points, actual BF16 encodings, repetitions, means, variation, timestamps, and API times. Its backend must be explicitly selected and verified. A deterministic CPU sweep checks only the API function; real resolution, drift, and noise require repeated device measurements with a documented measurement boundary.

Before physical measurements, manufacturer approval of ranges, dimensions, and driver must be obtained. Measurements should then particularly cover the key differences actually used, points near equality, forward/reverse sweeps, and interspersed time-stamped references. Raw values and error status must be preserved; a single Gaussian fit is insufficient if outliers, channel differences, or correlations determine the decision.

15.3 Fair digital and hybrid comparisons

The CPU baseline provides an initial performance reference. A GPU comparison must produce the same stable outputs and separately measure device-internal sorting time and total time including host/device transfers. Synchronization, warm-up, data type, batch size, and library must be recorded. An unstable implementation or one returning only top-k would perform a different task and must be labelled accordingly.

An NPU comparison first needs calibrated accuracy, then complete runtime and energy. Direct electronic sign decisions, the linear SDK path, and a confirmed cascaded nonlinearity form separate comparison cases. This is necessary to determine whether a photonic step saves work or merely re-encodes an already available decision.

15.4 Keeping the thesis viable without hardware access

The next priority is quantitatively supported parameterization of the shared model or, if such data remain unavailable, a clearly bounded sensitivity analysis over documented assumption ranges. A supplementary GPU baseline follows if a platform is available, alongside resource analysis with explicit conditions for a possible advantage.

A rigorous thesis remains possible without hardware: it can establish mathematical suitability, software functionality, failure types, quality requirements, digital cost, and conditional system limits. Its conclusions must then not claim measured photonic acceleration, energy superiority, or physical noise robustness.

16. Overall assessment of the current state

The thesis compares Bitonic with Louri's full rank matrix under the unchanged complexity model C = (T,S,H,D). Source equations, tie rules, and rank numbering are implemented transparently. The full matrix has been checked in the official CPU SDK as linear pair processing with electronic bias, thresholding, and rank construction.

Without noise, both methods agree with stable digital sorting. Under the explicitly assumed disturbances, they differ substantially: Bitonic preserves the record permutation, whereas inconsistent matrix decisions and disturbed self-comparisons can invalidate rank placement. These failures remain visible. Conditional ranking quality does not replace the validity rate.

The nonlinear feasibility study provides a reproducible precision finding: 40 false affine 8-bit ties. An advantage on actual photonic hardware remains unresolved. Next steps concern supported noise and capacity parameters, a fair GPU comparison, and, where possible, measurements of complete data paths. The documented modelling and CPU validation remain useful even without hardware access.

Appendix A. References and their roles

The following entries cover the works investigated in the two presentations, the supplied complete documentation, and the attached literature collections. Each entry states its role in the thesis. Individual PDFs, embedded original texts, analysis notes in compilations, and supplementary official documentation are distinguished. Mention in a compilation does not imply independent reproduction of the described hardware.

[1] Peter L. McMahon (2023). **The physics of optical computing**.

doi.org/10.1038/s42254-023-00645-5

Source basis: Individual PDF 18; supplied arXiv version 2308.00088. **Role:** Physical advantages, limitations, and system integration of optical computation; not a sorting benchmark.

[2] Andrés Macho-Ortiz, Daniel Pérez-López, José Azaña, José Capmany (2023; preprint 2022). **Analog Programmable-Photonic Computation**.

doi.org/10.1002/lpor.202200360

Source basis: Individual PDF 25. **Role:** Anbits, programmable linear gates, and feedback as APC foundations; classical optical processing, not a Q.ANT API specification.

[3] C. D. Thompson (1979). **Area-Time Complexity for VLSI**.

doi.org/10.1145/800135.804401

Source basis: Individual PDF 24. **Role:** Physical area-time trade-offs and communication; the DFT bound is not adopted as a sorting bound.

[4] H. J. Caulfield (1991). **Space-Time Complexity in Optical Computing**.

doi.org/10.1007/BF01937172

Source basis: Individual PDF 19. **Role:** Spatial, temporal, and fan-in resources limit simple claims of constant optical execution time.

[5] Yandong Li, Francesco Monticone (2025). **The spatial complexity of optical computing: toward space-efficient design**.

doi.org/10.1038/s41467-025-63453-8

Source basis: Individual PDF 17. **Role:** Space requirements of linear operators and nonlocal communication; applicability to complete nonlinear sorting is limited.

[6] K. E. Batcher (1968). **Sorting Networks and Their Applications**.

doi.org/10.1145/1468075.1468121

Source basis: Individual PDF 20. **Role:** Basis of fixed Bitonic stages and comparison counts; not a physical timing model.

[7] Charles W. Stirk, Ravindra A. Athale (1988). **Sorting with optical compare-and-exchange modules**.

doi.org/10.1364/AO.27.001721

Source basis: Individual PDF 07; also sorting collection 05. **Role:** Spatial C&E pipeline, implementation variants, and regeneration; starting point for the active Bitonic adaptation.

[8] F. R. Beyette, Jr., P. A. Mitkas, S. A. Feld, C. W. Wilmsen (1994). **Bitonic sorting using an optoelectronic recirculating architecture**.

doi.org/10.1364/AO.33.008164

Source basis: Individual PDF 02. **Role:** Reuse of smart-pixel arrays; distinguishes complete response time from output interval.

[9] M. P. Y. Desmulliez et al. (1995). **Perfect-shuffle interconnected bitonic sorter: optoelectronic design**.

doi.org/10.1364/AO.34.005077

Source basis: Individual PDF 06. **Role:** Perfect shuffle, control masks, and bit-serial cycles; not an additional modern SDK hardware experiment.

[10] Ahmed Louri, James A. Hatch, Jr., Jongwhoa Na (1995). **Constant-time parallel sorting algorithm and its optical implementation using smart pixels**.

doi.org/10.1364/AO.34.003087

Source basis: Individual PDF 03. **Role:** Broadcast-and-compare, rank construction, and ties; constant number of wide layers with quadratic resources.

[11] Kenji Mizutani et al. (2025; online 2024). **In-network stable radix sorter using many FPGAs with high-bandwidth photonics [Invited]**.

doi.org/10.1364/JOCN.530695

Source basis: Individual PDF 04. **Role:** FPGA/HBM sorting with photonic links; eight-FPGA measurements and a separate scaling projection.

[12] Sufi R. Ahmed et al. (2025). **Universal photonic artificial intelligence acceleration**.

doi.org/10.1038/s41586-025-08854-x

Source basis: Individual PDF 16. **Role:** Integrated hybrid accelerator illustrating digital peripherals and conversion costs; not a Q.ANT system.

[13] Xiansong Meng et al. (2025). **Digital-analog hybrid matrix multiplication processor for optical neural networks**.

doi.org/10.1038/s41467-025-62586-0

Source basis: Individual PDF 08. **Role:** Hybrid precision improvement for matrix operations; additional digital work must be counted.

[14] Zichao Zhao et al. (2025). **Efficient hardware error correction with hybrid on-offline configuration algorithm for optical processor**.

doi.org/10.1038/s42005-025-02247-2

Source basis: Individual PDF 09. **Role:** Calibration through offline/online configuration; motivates separate treatment of systematic errors.

[15] Huize Li, Dan Chen, Tulika Mitra (2025). **Hybrid Photonic-digital Accelerator for Attention Mechanism (HyAtten)**.

arxiv.org/abs/2501.11286

Source basis: Individual PDF 10; preprint. **Role:** Selective digital processing with limited analog precision; a possible direction, not implemented in the sorter code.

[16] Yaowen Hu et al. (2025). **Integrated lithium niobate photonic computing circuit based on efficient and high-speed electro-optic conversion**.

doi.org/10.1038/s41467-025-62635-8

Source basis: Individual PDF 11. **Role:** TFLN-based modulation and computation; no evidence of freely cascaded Q.ANT functions.

[17] Hailong Gong et al. (2026). **LightMat-HP: A Photonic–Electronic System for Accelerating General Matrix Multiplication With Configurable Precision**.

arxiv.org/abs/2604.12278

Source basis: Individual PDF 12; arXiv v1, 14 April 2026. **Role:** Low-bit photonic partial products and digital reconstruction; the DOI placeholder in the supplied version is not used as publication evidence.

[18] Hajime Sakata (2001). **Photonic analog-to-digital conversion by use of nonlinear Fabry–Perot resonators**.

doi.org/10.1364/AO.40.000240

Source basis: Individual PDF 13. **Role:** Nonlinear optical conversion as an alternative component; does not provide parameters for the hardware used here.

[19] Nathaniel Nauman, James Robinson, Yuyang Wang, Kaylx Jang, Xiang Meng, Keren Bergman (2025). **Photonic Analog-to-Digital Architecture for Accelerating Multiply-Accumulate Operations**.

doi.org/10.1364/OFC.2025.W3D.5

Source basis: Individual PDF 14; OFC 2025, W3D.5, additionally checked with the publisher. **Role:** Photonic ADC architecture for MAC; motivates the converter interface, not a Q.ANT measurement.

[20] Jing Xu et al. (2025). **Progress in silicon-based reconfigurable and programmable all-optical signal processing chips**.

doi.org/10.1007/s12200-025-00154-6

Source basis: Individual PDF 15. **Role:** Review of programmable signal processing and regeneration; functional diversity requires specific components.

[21] Raúl López-March, Andrés Macho-Ortiz, Francisco Javier Fraile-Peláez, José Capmany (2026). **Noise in analog programmable-photonic computation**.

arxiv.org/abs/2604.24541

Source basis: Individual PDF 22; arXiv v1, 27 April 2026. **Role:** Physical noise sources and error propagation; example values belong to that PIP arrangement and are not a Q.ANT calibration.

[22] Niklas Bahr, Daniel Steinmeyer, Wolfram Pernice (2026). **Energy Efficiency in Analog Photonic Processors: Conversions and Losses at Scale**.

doi.org/10.23919/ISC.2026.11520497

Source basis: Individual PDF 23; image-based version, additionally read using OCR. **Role:** System cost model for conversion, weights, memory, and losses; excludes digital control logic and does not establish sorting energy use.

[23] Hyoseok Park, Yeonsang Park (2026). **PRISM: Photonic Similarity Engine for KV Cache Block Selection in Long-Context LLM Inference**.

arxiv.org/abs/2603.21576

Source basis: Individual PDF 21; v2 of 25 March 2026 with supplement. **Role:** Photonic scoring and electronic selection; application motivation for recall, not complete stable integer sorting.

[24] Morris Rubinoff (1953). **Analogue vs. Digital Computers—A Comparison**.

ieeexplore.ieee.org/document/4051190

Source basis: Research compilation 01, starting on p. 3. **Role:** Historical distinction between analog and digital computing principles; not a quantitative modern hardware baseline.

[25] Guo Qi Zhang, Alfred van Roosmalen (eds.) (2009). **More than Moore: Creating High Value Micro/Nanoelectronics Systems**.

doi.org/10.1007/978-0-387-75593-9

Source basis: Research compilation 01, p. 2; book reference. **Role:** Technological context beyond transistor scaling; not a separate sorting or photonics experiment.

[26] H. John Caulfield, Shlomi Dolev (2010). **Why future supercomputing requires optics**.

doi.org/10.1038/nphoton.2010.94

Source basis: Research compilation 01, starting on p. 14. **Role:** Motivation for optical communication and parallelism; a forward-looking argument, not evidence for this workload.

[27] Francesco Morichetti (2025). **High-performance analog signal processing with photonic integrated circuits**.

doi.org/10.1038/s41377-025-01806-0

Source basis: Research compilation 01, starting on p. 18; commentary. **Role:** Context for integrated analog signal processing; the commentary and primary demonstrators it discusses remain distinct.

[28] H. Zhou et al. (2022). **Photonic matrix multiplication lights up photonic accelerator and beyond**.

doi.org/10.1038/s41377-022-00717-8

Source basis: Research compilation 01, starting on p. 19; presentation literature. **Role:** Review of matrix multiplication using spatial, interferometric, and wavelength-based methods; context for MVM.

[29] P. Bruce Berra, Arif Ghafoor, Pericles A. Mitkas, Slawomir J. Marcinkowski, Mohsen Guizani (1989). **The Impact of Optics on Data and Knowledge Base Systems**.

ieeexplore.ieee.org/document/43408

Source basis: Research compilation 01, starting on p. 28. **Role:** Early context for optical database operations and data movement; not a directly comparable current measurement.

[30] Yinyi Liu et al. (2022). **A Reliability Concern on Photonic Neural Networks**.

doi.org/10.23919/DATE54114.2022.9774661

Source basis: Research compilation 01, starting on p. 45. **Role:** Architecture-specific reliability and crosstalk; not a universal mode-count or noise limit for Q.ANT.

[31] Bibliographic entry from the research compilation (2025). **Integrated PCM-based DAC/ADC Design for Energy-Efficient Photonic Computing Systems**.

doi.org/10.1117/12.3066474

Source basis: Research compilation 01, starting on p. 108; analysis notes. **Role:** PCM-based conversion as another approach; not separately reproduced. Percentage savings in the notes are not adopted as our own findings or Q.ANT results.

[32] Nikolaos Farmakidis, Bowei Dong, Harish Bhaskaran (2024). **Integrated photonic neuromorphic computing: opportunities and challenges**.

doi.org/10.1038/s44287-024-00050-9

Source basis: Research compilation 01, starting on p. 363. **Role:** Systematic classification of hybrid neural photonics and interfaces; motivates complete execution boundaries.

[33] M. A. Al-Qadasi, L. Chrostowski, B. J. Shastri, S. Shekhar (2022). **Scaling Up Silicon Photonic-Based Accelerators: Challenges and Opportunities**.

doi.org/10.1063/5.0070992

Source basis: Research compilation 01, starting on p. 377; preprint 2109.08025. **Role:** Optical and electrical link budgets, scaling, and energy costs; parameters must match the specific setup.

[34] G. Giamougiannis et al. (2023). **Analog nanophotonic computing going practical: silicon photonic deep learning engines for tiled optical matrix multiplication with dynamic precision**.

doi.org/10.1515/nanoph-2022-0423

Source basis: Research compilation 01, starting on p. 393. **Role:** Tiling and dynamic precision as system methods; no automatic guarantee of exact comparison decisions.

[35] Qingyi Zhou et al. (2026). **Quantum Nonlinearity for Optical Neural Computing**.

arxiv.org/abs/2601.01690

Source basis: Research compilation 01, starting on p. 413; current arXiv entry additionally checked. **Role:** Quantum-emitter nonlinearity in a different architecture; not equivalent to classical APC or an available Q.ANT cascade path.

[36] Q.ANT GmbH (SDK 2.3.1). **Q.ANT Native Computing Toolkit: pinned source version**.

github.com/Q-ANT-GmbH/qant_native_computing_toolkit/tree/72a2d99f10240b6df3c6d0f636dfa0e2b5d38902

Source basis: Official SDK source version; checksums and CPU package included in the project. **Role:** API contracts, BF16 formats, host ReLU, and the CPU/hardware distinction; not a fully open hardware driver.

[37] Q.ANT GmbH (accessed 7 October 2026). **Photonic Computing: product information**.

qant.com/photonic-computing/

Source basis: Official product page; supplemented by the June 2026 Generation 2 brochure. **Role:** Manufacturer statements on interface, performance, and product; not our own comparator, transfer-characteristic, or sorting measurements.

[38] NumPy Developers (Documentation 2.3). **numpy.argsort and numpy.take_along_axis**.

numpy.org/doc/2.3/reference/generated/numpy.argsort.html

Source basis: Official API documentation; installed benchmark version 2.3.5. **Role:** Stable indirect sorting and gathering original values; numerical timing results come exclusively from our saved measurements.

[39] SciPy Developers (Documentation). **scipy.stats.kendalltau**.

docs.scipy.org/doc/scipy/reference/generated/scipy.stats.kendalltau.html

Source basis: Official statistics documentation; independent implementation check. **Role:** Definition and tie correction of Kendall-τ-b; does not replace separate validity and stability metrics.

[40] Nikita Stroev, Natalia G. Berloff (2023). **Analog Photonics Computing for Information Processing, Inference and Optimisation**.

arxiv.org/abs/2301.11760

Source basis: Presentation P2, p. 25; arXiv entry additionally checked. **Role:** Review of analog photonic platforms, optimization, and inference. The presentation must not infer universally error-free physical sorting from it.

Appendix B. Source corpus and result provenance

B.1 Supplied documents

The consolidation examined both presentation PDFs, the previous detailed report, and 25 supplied research PDFs. The two extensive literature/sorting compilations contain additional original texts or notes and overlap with individual PDFs. The following inventory documents files, not 25 independent research findings. Original full texts are not copied into the public repository.

Table 22. Supplied files reviewed

ID	Document	Pages
01	Research compilation: Analog Photonic Computing, hybrid architectures, and sorting	460
02	Bitonic sorting using an optoelectronic recirculating architecture	9
03	Constant-time parallel sorting algorithm and its optical implementation using smart pixels	10
04	In-network stable radix sorter using many FPGAs with high-bandwidth photonics [Invited]	12
05	Collection of optical sorting methods	51
06	Perfect-shuffle interconnected bitonic sorter: optoelectronic design	14
07	Sorting with optical compare-and-exchange modules	6
08	Digital-analog hybrid matrix multiplication processor for optical neural networks	11
09	Efficient hardware error correction with hybrid on-offline configuration algorithm for optical processor	9
10	Hybrid Photonic-digital Accelerator for Attention Mechanism (HyAtten)	7
11	Integrated lithium niobate photonic computing circuit based on efficient and high-speed electro-optic conversion	11
12	LightMat-HP: A Photonic–Electronic System for Accelerating General Matrix Multiplication With Configurable Precision	28
13	Photonic analog-to-digital conversion by use of nonlinear Fabry–Perot resonators	9
14	Photonic Analog-to-Digital Architecture for Accelerating Multiply-Accumulate Operations	3
15	Progress in silicon-based reconfigurable and programmable all-optical signal processing chips	53
16	Universal photonic artificial intelligence acceleration	8
17	The spatial complexity of optical computing: toward space-efficient design	11
18	The physics of optical computing	31
19	Space-Time Complexity in Optical Computing	6
20	Sorting Networks and Their Applications	8
21	PRISM: Photonic Similarity Engine for KV Cache Block Selection in Long-Context LLM Inference	27

ID	Document	Pages
22	Noise in analog programmable-photonic computation	27
23	Energy Efficiency in Analog Photonic Processors: Conversions and Losses at Scale	9
24	Area-Time Complexity for VLSI	8
25	Analog Programmable-Photonic Computation	15

IDs 01–25 correspond to the supplied research files. Original filenames and hashes are in sources.json.

B.2 Result collections used

Table 23. Current result collections used

ID / directory	Contents	Evidence limitation
D1: results/reference	All Bitonic/Louri outputs, accuracy, seeds	Assumed independent cell noise
D2: results/sdk	All noiseless SDK outputs and pair counts	Official CPU backend, no hardware measurement
D3: results/periodic	Exhaustive periodic pair check	Comparison function, not a matrix sorter
D4: results/ranking_quality	τ, recall, inversions by key distance	Conditional ranking quality requires valid outputs
D5: results/common_noise	Pair and sorting controls in common units	Synthetic Float64 models
D6: results/cpu_baseline_20261007	Measured CPU timings and environment	Measurement platform of 7 October 2026
D7: results/affine	All scores and errors of the affine pair function	No data path without a host round trip demonstrated
D8: results/louri	Source equations and threshold controls	Isolated pair cases hold the diagonal exact
results/resources	Analytical call and logical-buffer counts; SDK count check	No measured device capacities
data/inputs.npz	24 datasets with 1,000 lists each	Unchanged shared input corpus

report_assets/sources.json identifies the supplied research sources. report_assets/evidence.json binds all result files currently used to the report through SHA-256. Run metadata preserve source hashes and environments at execution time. They do not replace physical device characterization.

Appendix C. Terminology and reading guide

Table 24. Key terminology

Term	Meaning in this report
APC / anbit	Analog Programmable-Photonic Computation / two-component classical optical information unit
MVM / GEMM	Matrix-vector multiplication / general matrix-matrix multiplication
C&E	Compare-and-exchange: compare and, if necessary, exchange original records
T / physical time	Total scalar real-RAM-like work / separately measured or modelled seconds
S / H / D	Live logical words / installed processing positions / unregenerated optical depth
Ω / Θ / O	Lower / matching lower and upper / upper asymptotic bound
Δ / η / σ	Key or signal grid / dimensionless noise factor / standard deviation in signal units
ADC / DAC / ENOB	Analog-to-digital converter / digital-to-analog converter / effective bit count from a defined measurement
BF16	Bfloat16 storage/interface format; no guarantee of effective analog precision
tcos / CPU cosine	Device-specific periodic function / separately investigated CPU substitute
Zero band / reference	Tolerated tie interval / subtracted output value for d = 0
Validity / stability	Output every original record exactly once / preserve original order of equal keys
Kendall-τ-b / Recall@k	Tie-corrected rank agreement / fraction of correctly selected top-k elements
RIN / O/E/O	Relative intensity noise density / optical-electronic-optical transition

Further methodological details are in docs/METHOD.md, docs/RANKING_QUALITY.md, docs/COMMON_NOISE_MODEL.md, docs/CPU_BASELINE.md, and docs/QANT_MEASUREMENT_PLAN.md. docs/CODE_GUIDE.md introduces the modules. This report connects those details into a continuous account; executable files and saved data remain the authoritative basis for reproduction.